

JPMC – Anvar Zokhidov

Report

Important! In the following paragraphs I will be providing all my thoughts, insights, explanations and the results based on my own experience from previous companies. Most of these best practices I obtained came from 'Tenge Bank' and through the collaboration with 'Ernst & Young (EY)' that were our consultants.

Experts say that one of the most important aspects in working with a Data Science/ML project is not training a model but working with data that takes around 80% of your time. It does not matter how complicated model you train or tune, if the dataset is poor than the model will perform poor too. So, it is essential to prepare a great dataset from raw data in possession followed by cleaning it. However, unlike school/interview in real world no one will provide you with even raw data. In my case, along with using data from our internal data warehouse we collaborated with several external data providers. It is essential to clearly state what data you want, its type/format, size as well as further support from them. This is even more important than data cleaning because if the raw data has been provided in a desired state all the further dataset preparation, feature engineering and model building will go smoothly.

The best way to know beforehand how to approach any new project is:

- Talk to people from your team/department/similar background. At almost any company or firm there will be people that have already worked with a similar project, data, tech or a provider. It works like magic but they will have the answer you need.
- Go to another floor and talk to people from other departments.
- Therefore, networking within the company is essential. It will save time and give best practices that will save huge amount of money to the company and solve your tasks.

The dataset I was provided for this project contains “weighted census data extracted from the 1994 and 1995 Current Population Surveys conducted by the U.S. Census Bureau.”

For the project, I used 'Python' and related libraries. After extracting the dataset and the corresponding columns in a jupyter notebook (perfect to show/explain this project), we see that it has 199.523 rows and 42 columns. Each row represents a person and their demographic & employment related features that we use as independent variables/predictors to train a model to predict the target variable 'label' that represents whether a person has an income over or less than \$50k.

num persons worked for employer	family members under 18	country of birth father	country of birth mother	country of birth self	citizenship	own business or self employed	fill inc questionnaire for veteran's admin	veterans benefits	weeks worked in year	year	label
0	Not in universe	United- States	United- States	United- States	Native- Born in the United States	0	Not in universe	2	0	95	- 50000.
1	Not in universe	United- States	United- States	United- States	Native- Born in the United States	0	Not in universe	2	52	94	- 50000.

How exactly will the retail business use the model for marketing purposes?

- It is the question to be verified by a data scientist from stakeholders via frequent communication. Because the goal is not to build a good model to impress everyone, but build a product that will eventually generate revenue directly or indirectly.

We start cleaning the dataset (initial procedure before training a model) by looking at its data type, missing values and unique values of each variable.

	Data Type	Null Counts	Non-Null %	Unique Values
age	int64	0	100.000000	91
class of worker	object	0	100.000000	9
detailed industry recode	int64	0	100.000000	52
detailed occupation recode	int64	0	100.000000	47
education	object	0	100.000000	17
wage per hour	int64	0	100.000000	1240
enroll in edu inst last wk	object	0	100.000000	3
marital stat	object	0	100.000000	7
major industry code	object	0	100.000000	24
major occupation code	object	0	100.000000	15
race	object	0	100.000000	5
hispanic origin	object	874	99.561955	9
sex	object	0	100.000000	2

In the dataset int64 & float64 are numerical values that we can directly fit to our model for the sake of this project only. In other cases such as banks (Since JPMC is a bank) continues values usually are divided into ranges called bins to be further used as categories to build a score card. Some numerical categorical features might also need to be coarse and fine classed (increasing # of bins or decreasing). The reason might be algorithmicly/statisticly or business strategy based. Object datatypes here are strings (text) that will be one-hot encoded later (transformed into 0&1 vectors).

The screenshot above shows that there are 874 nulls/missing values for the 'hispanic origin' variable. Reasons:

- Not recorded
- People did not answer this question
- Technical error occurred
- Can be anything

How to solve:

- Delete rows if dataset is large enough and/or ratio of nulls is low (0.04 in our case)
- Replace with mean, median, mode (median usually recommended as if there are outliers your mean could fly to the moon:)
- Replace with 'Unknown', 0 or huge negative/positive number et etc. This is tricky as each one of them could represent some info. Ex: some variable 'defaulted n times' = Null turned to 0 might make him or her a good customer. What if the customer actually defaulted a number of times and this is a technical error?

- Stay consistent with company and country regulations. Race, origin, gender and other similar variables might be not allowed to use for model training in order not to make it biased.

We replaced them with 'Unknown' as it will serve as a separate category. There are interesting entries such as 'not in universe'. We use them as a separate category too.

```
# The sum of the weight column values should represent the US population at the time of record
df['weight'].sum()

np.float64(347245892.46999997)
```

The sum of the weight column values should represent the US population at the time of record = 347.245.892. This is done to show what number of people represents each group they belong to. It is hard to record the whole population.

Then we dropped duplicate rows as they might add unnecessary impact to the model.

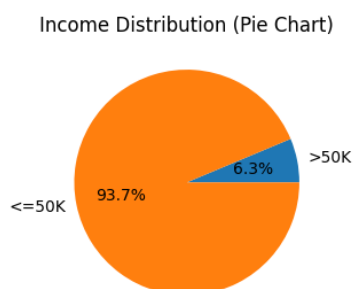
Dataset shape "before" dropping duplicates (rows, columns) = (199523, 42).

Dataset shape "after" dropping duplicates (rows, columns) = (196294, 42).

3229 rows that were duplicates were dropped. In our case it is 1.6% of the whole dataset. So, we can go further as the dataset has barely changed in numbers. However, in cases when the number drops significantly further exploration is needed within the dataset.

label	-50000.	+50000.
target	0	1

We change the column name 'label' to 'target', and -50000 to 0 and +50000 to 1. The goal of the model is to be able to correctly classify between them.

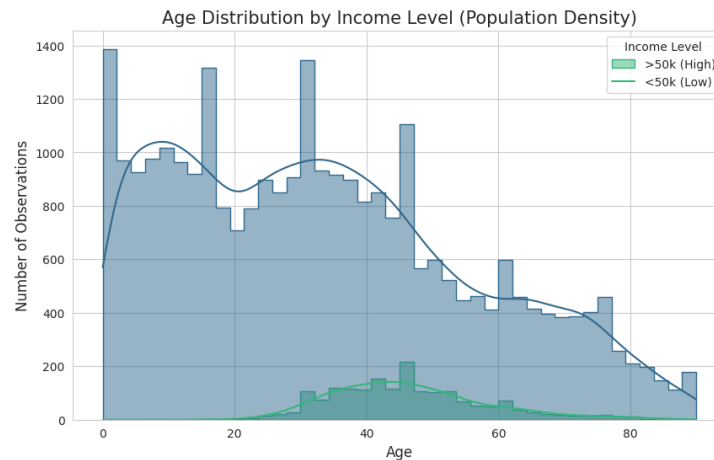


The image above illustrates income distribution:

(Income more than 50k): 6.3%

(income less than 50k): 93.7%

We can see that the dataset is unbalanced with 93.7% having less than 50k and 6.3% more than 50k income. We should take it into account while training the model.



Age is considered one of the most important and default variables as it has a predictive value for the target and present in almost every dataset. The image shows:

Average age for individuals earning <\$50k: 33.8 years

Average age for individuals earning >\$50k: 46.5 years

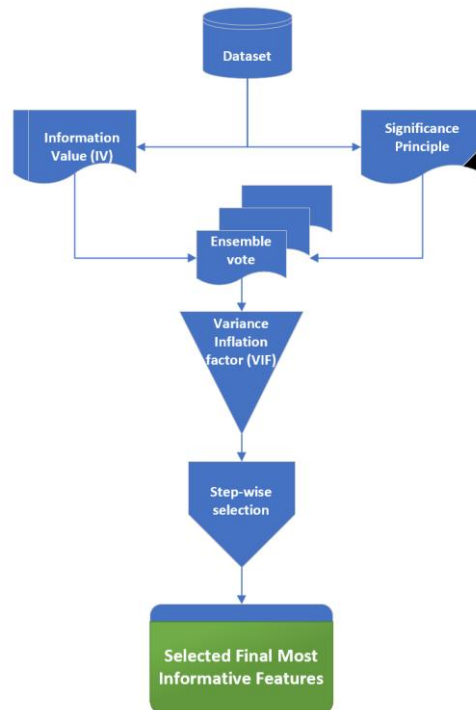
Automatic Feature/variable Selection

While training a model or performing tasks we have to choose which features/columns to use from our dataset. Reasons: 1) Not all the features are informative enough to contribute to the model 2) It makes model more robust and it will become slow to train 3) Harder to interpret and explain to stakeholders 4) Harder to interpret for regulators.

Therefore, following is a set of algorithms that automatically selects the best features from the dataset to include in the model training. Each model selects a set of informative features and the final result (selected features) will be their ensemble vote.

The architecture below shows the workflow of the feature selection. The dataset is fed into the Information Value and Significance Principal algorithms. They output most informative features that contribute the most to predicting the target (if a person has >50k or <50k). Their ensemble vote, that is, if both algorithms' outputs have the same variable then we pick it up. Further the selected features are fed into the Variance Inflation Factor (VIF) algorithm that check and drops variables if multicollinearity is detected. Finally, the Step-wise selection algorithm filters out the variables once more and outputs a final selected set of most informative variables to predict the target.

- The workflow is this way as VIF runs slow compared to the previous algorithms. The previous ones decrease the number of variables that increases the inference speed of VIF. Step-wise selection is the slowest one, therefore it comes at the end as a minimum set of features being fed.



Information Value (IV)

- Definition: IV is a numerical value that shows the "predictive power" of a variable in relation to a binary target (Income >50k vs <50k). It is calculated using the Weight of Evidence (WoE). **WoE is another technique that can be used to build a model or a scorecard. I have a published article on that that will be included in the report.**
- Business logic: You can rank variables: <0.02 Unpredictive | 0.02 to 0.1 Weak | 0.1 to 0.3 Medium, 0.3 to 0.5 Strong | >0.5: Suspicious (potentially too good to be true), maybe data leakage
- WoE & IV is very convenient as it can deal with numerical values, strings and even nulls. Also, it brings non-linearity to the model.

Significance Principal

- Definition: Uses Lasso Regression for feature selection
- Business logic: Shrink non informative features to 0 and eliminate

Variance Inflation Factor (VIF)

- Definition: VIF measures multicollinearity—how much one feature can be predicted by the other features in the model.
- Business Logic: High multicollinearity makes a model's coefficients unstable. Ex: if "Age" and "Year of birth" are both in the model, they tell the same story.
- A VIF of 5 and above is suspicious and of above 10 should be removed.

Stepwise Selection

- Definition: Iterative process that adds or removes features one by one based on their statistical significance.
- Types:
 - Forward - Starts with no variables and adds the best ones.
 - Backward: Starts with all variables and removes the weakest ones.
- Business Logic: Gives the most accuracy but runs extremely slowly especially if there are many variables/features.

In Total 13 variables has been selected from the algorithms to train the model:

weeks worked in year | detailed occupation recode | detailed industry recode | num persons worked for employer | age | education | detailed household and family stat | detailed household summary in household | tax filer stat | family members under 18 | major occupation code | major industry code | class of worker

Usually, business end up having a massive data pool. Data Scientists perform feature engineering on them and create datasets or buy datasets from external providers such as Credolab. They might have thousands of features. The feature selection algorithm pipeline introduced here automatically selects most valuable/informative features from every new dataset.

Training a classifier

There is a big option of algorithms to train a classifying model. My choice is – Logistic Regression. From my previous work experience, although Logistic Regression is a base and relatively simple model it is most commonly deployed in production. Business idea and why?

- Stakeholders understand the basic logic behind Logistic Regression and can see why the customer was classified that way. Though Logistic Regression should usually be converted into something more interpretability such as a score card where a customer is assigned points. The sum of the points can further be transformed into 'A+', 'A', 'A-' 'D'. Business can write rules to each letter. This way one can explain the logic behind the decision made, even to non-technical managers. Ex: Customer/Clients lies within 'D', so we reject them.
- Regulators have best practices to assess Logistic Regression. Ex: each variable has been converted into dummies (one-hot 0&1). A Coefficient * 0 = 0, a coefficient * 1 = coefficient.
- It is stable. Usually datasets have a 'date' column based on which Data Scientists/ml engineers do feature engineering and "Out of time" testing. "Out of time" testing is crucial (unlike classical train, cross validation, test splits) as it shows how good the model does in a later time period. Logistic

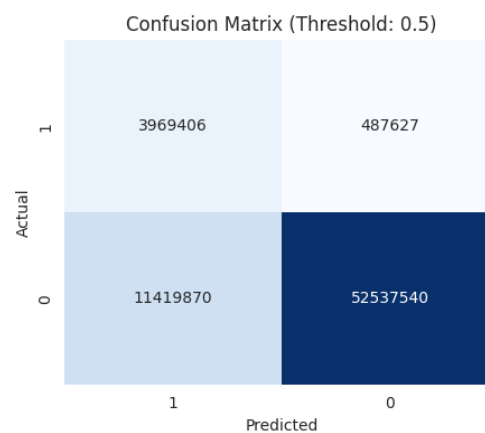
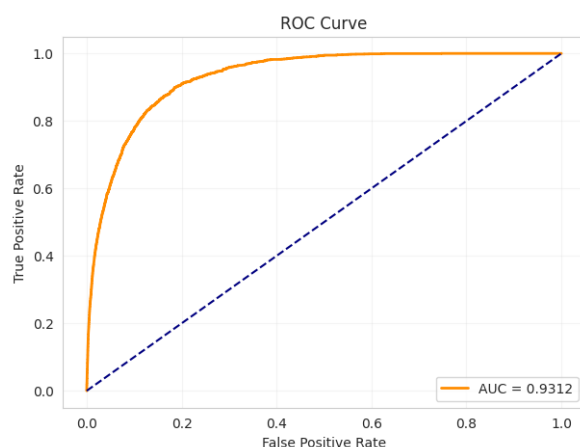
Regression showed stability up to 3 months in my previous bank job experience before dropping its performance. Other algorithms dropped in a few weeks.

Training: Numerical features were normalized with StandardScaler and categorical variables & strings with OneHotEncoder (Other options: One hot encode everything or use WoE). Label encoding is usually not recommended as it might bring ordering relationship. The 'weight' column was included during training not as a feature column but as the 'classifier__sample_weight' parameter to reflect the true population size. The model was trained with a 5 fold cross validation and grid search that automatically selected the best hyperparameters. Classes are weighted to deal with the data imbalance. Train size = 80%, test size = 20% with label classes evenly distributed (stratified by target y). A dataset size defines a train/test split. Here it is large enough for 80~20%.

Evaluation:

Best Hyperparameters: {'classifier__C': 0.01, 'classifier__class_weight': 'balanced', 'classifier__penalty': 'l2', 'classifier__solver': 'liblinear'}

Metrics: Accuracy: 0.8260 F1-Score:0.3914 ROC-AUC:0.9312 Gini:0.8624



Since the data is unbalanced, accuracy is not representative, F1-score is not high but is OK for this unbalanced data. I would look more at the ROC-AUC&Gini scores as well as confusion matrix. ROC-AUC&Gini are usually default metrics for banks with the Gini ($2 \times \text{ROC-AUC} - 1$) of 55 – 70% being considered good to deploy for demographics and income data. With real data that is usually messed up scores above 90% might actually be a bad sign. Probably data leakage occurred, or you hit the jackpot finding an extremely representative feature which is practically unreal. The feature cannot be that representative and be present for each customer. Otherwise, with this feature no data scientists and ml engineers are needed. You can just put several thresholds on this only feature/variable and make decisions.

Confusion matrix is the second metric I would use during production. Due to economic situation, new strategy or push from top managers, business might ask you to calibrate the model. Ex: providing marketing for less wealthy clients (<50K) is better than not providing it to wealthy customers (>50K) as profit is made out of people who can pay. So, we might want to set the threshold a bit lower, say 30%, meaning anyone who is predicted more than 30% confidence by the model will be considered wealthy.

We can then start working with those customers. Vice versa is also applicable, if marketing should be provided only to customers who we are sure are wealthy. This way we increase the threshold. This becomes a flexible tool/system to calibrate business.

Top 5 High-Income Predictors:		
	Feature	Weight
16	education_Doctorate degree(PhD EdD)	3.048786
20	education_Prof school degree (MD DDS DVM LLB JD)	2.889046
19	education_Masters degree(MA MS MEng MEd MSW MBA)	2.106451
81	major occupation code_Farming forestry and fis...	2.081258
106	major industry code_Mining	2.026707
Top 5 low-Income Predictors:		
	Feature	Weight
125	class of worker_Without pay	-2.382402
47	detailed household and family stat_Other Rel 1...	-2.404581
80	major occupation code_Executive admin and mana...	-2.629415
15	education_Children	-3.700874
18	education_Less than 1st grade	-5.161412

Above are variables that predict high income (positive) and low income (negative). This is why Logistic Regression is comprehensible. The higher value coefficients/weights greatly classify and generalize to new unseen data. Logic: If someone has a PhD degree then there is a higher chance the person earns more than 50K and algorithmically the variable gets multiplied by positive 3.048, and 'education_less than 1st grade gets multiplied by negative -5.16 pushing a prediction to be less than 50K.

I have trained other algorithms as well apart from Logistic Regression: Support Vector Machines, Random Forest, XGBoost, Neural Networks. Despite they showed better metric results I decided the main model would still be Logistic Regression due to the reasons stated above. SVM is not recommended as for large datasets (20k points and above) it will never stop running. Random forest is better as you don't have to normalize the variables. I recommend to use XGBoost or any other boosting algorithm for production. Usually, banks have both Logistic Regression as a base model and a Boosting for a more complicated task such as dealing with 'credit card transactional' data where complex patterns should be found. Best practice is to build a microservice with FastAPI and deploy a Boosting model there. Neural networks are 'must have' not 'must use'. Although business will highly unlikely deploy it at the moment but, eventually we will switch into neural networks. Furthermore, stakeholders will ask: 'Do we have AI and neural networks?' as the current AI trend asks this from everyone. So, this is future work.

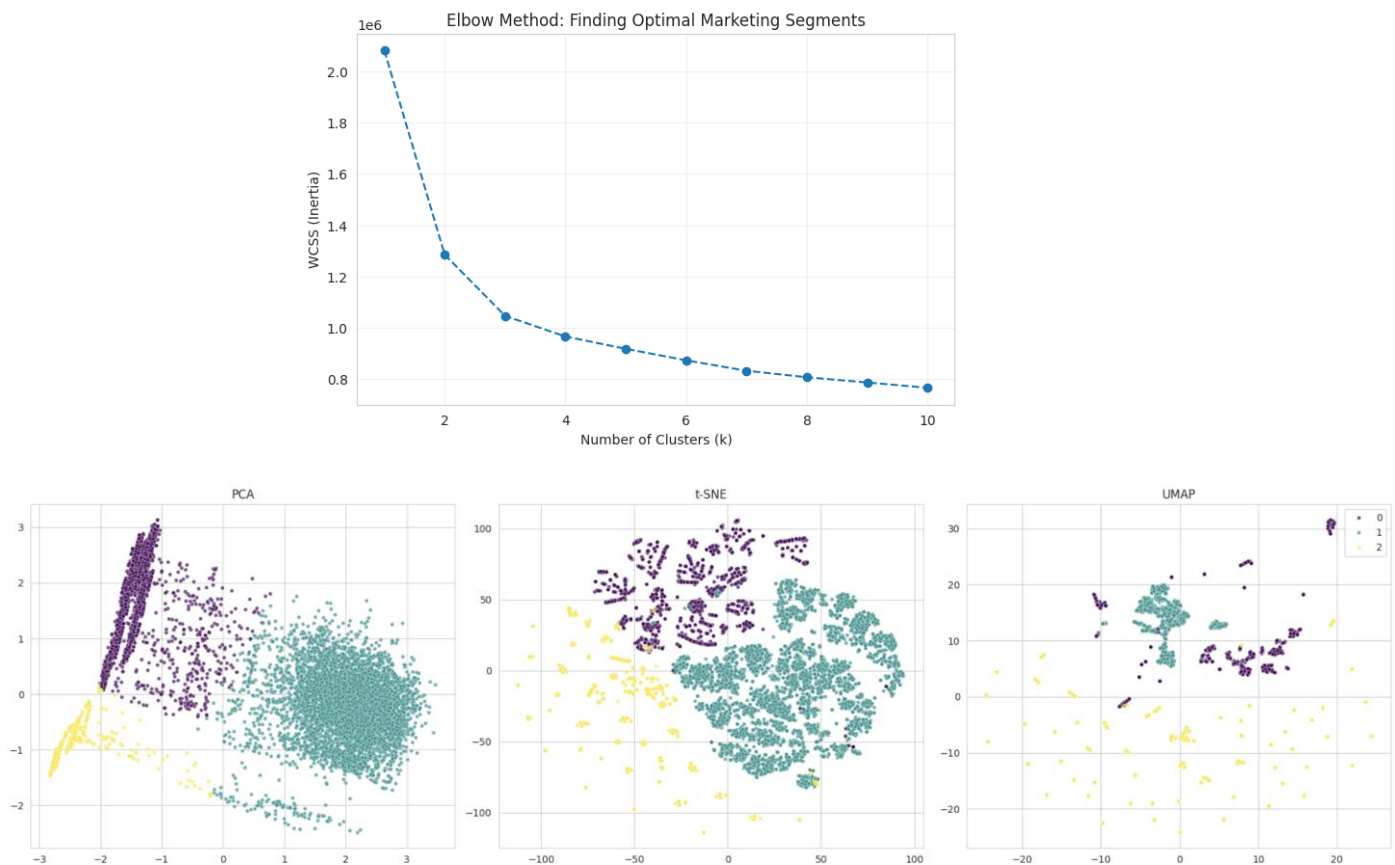
```
{ 'Random Forest': { 'Accuracy': 0.9365750528541226,
  'F1-Score': 0.4309872029250457,
  'ROC-AUC': np.float64(0.8909405397160319),
  'Gini': np.float64(0.7818810794320639)},
  'XGBoost': { 'Accuracy': 0.9464836088540207,
  'F1-Score': 0.4709141274238227,
  'ROC-AUC': np.float64(0.9305383004906257),
  'Gini': np.float64(0.8610766009812514)}}
```

```
Weighted Neural Network Results
Accuracy (Weighted): 0.9474
F1-Score (Weighted): 0.4691
ROC-AUC (Weighted): 0.9351
Gini (Weighted): 0.8702
```

Segmentation

I decided to use an unsupervised learning clustering algorithm K-means for market segmentation. The logic was to capture the general trend and patterns. Another way for segmentation that I did not use here is supervised learning with Weight of Evidence and Information Value (WoE & IV)

- Clustering Logic: Using K-Means clustering, I segmented the population based on the 126-feature transformed space (due to one hot encoding). I used the Elbow method to determine the optimal number of clusters K where the sharpest angle in the plot corresponds to K.
- However, from my former experience the Elbow method is a start followed by manual decisions. I used another unsupervised method called dimensionality reduction to reduce the dimension of the dataset to 2 (we can't visualize high dimensional data) otherwise. Algorithms used for that are: PCA, t-SNE and UMAP as they all provide different views to the data.



This is tricky. There is no 'the' optimal class number. 'An' optimal number can change based on observation, visualization and business goal. In this case, I chose K=3.

Market Segment Profiles								
	High_Earner_Ratio	age	weeks worked in year	education	major occupation code	tax filer stat	Population_Size	Pop_Percentage
Segment								
1	0.116401	38.558105	45.106230	High school graduate	Adm support including clerical	Joint both under 65	1.756624e+08	51.334306
0	0.018771	56.701378	3.212632	High school graduate	Not in universe	Nonfiler	8.052383e+07	23.531703
2	0.000000	8.175631	0.262047	Children	Not in universe	Nonfiler	8.600675e+07	25.133991

K-Means identified 3 distinct socio-economic life stages. **Segment 1** shows the primary workforce, **Segment 0** represents the senior/retired demographic, and **Segment 2** shows non-working age dependents.

Segment 1: "Target" (Main Workforce)

- Profile: Average age 38, working 45 weeks/year, High School graduates, typically Married (Joint filers).
- Income: Highest earning potential (11.6% earn >\$50k).
- Business Opportunity: This is our target clients being in peak and of 51% of the population.
- Marketing recommendation: Focus on "Family-sized" offerings and loyalty programs. More people in a family means more purchases.

Segment 0: "Seniors/Retirees" (Post-Career Demographic)

- Profile: Highest average age (56), low work intensity (3 weeks/year), and mostly "Non-filers."
- Income: Very low (1.8% high-earners), likely living on social benefits or fixed income.
- Business Opportunity: Have lower income, but are probably more loyal. If we get them, they will not leave us.
- Marketing recommendation: Focus on physical catalogs, senior-specific discounts and "comfort-focused" product lines.

Segment 2: "Future Market" (Youth & Dependents)

- Profile: Average age 8, almost no work history, and "Non-filers."
- Income: 0%. They do not work.
- Business Opportunity: No income but they ask their parents 'segment 1' to buy them products.
- Recommendation: Focus on toys through the parents who have income.

I would be happy to discuss more about the business aspects and opportunities within the project (and more) in the next steps of the interview. Thank you very much for your time!

Question: What are the regulation requirements for such a project from JPMC and client?

List of references to resources: I should say 'thank you' to my colleagues from the companies I worked before that gave me most of the real experience and knowledge: Tenge Bank, EY (Ernst & Young), Orange (France Telecom), The French National Centre for Scientific Research (CNRS). Online courses from Stanford, Coursera, Udemy, Udacity, IBM, AWS. Degrees/studies from 'The University of Texas at Dallas', 'Université Côte d'Azur', 'Tashkent University of Information Technologies'.

My articles for 'Score cards', 'Banking', 'Modeling':

<https://scientificbulletin.com/index.php/IJDSR/article/view/1115?articlesBySimilarityPage=6>

<https://scientificbulletin.com/index.php/IJDSR/article/view/1059>